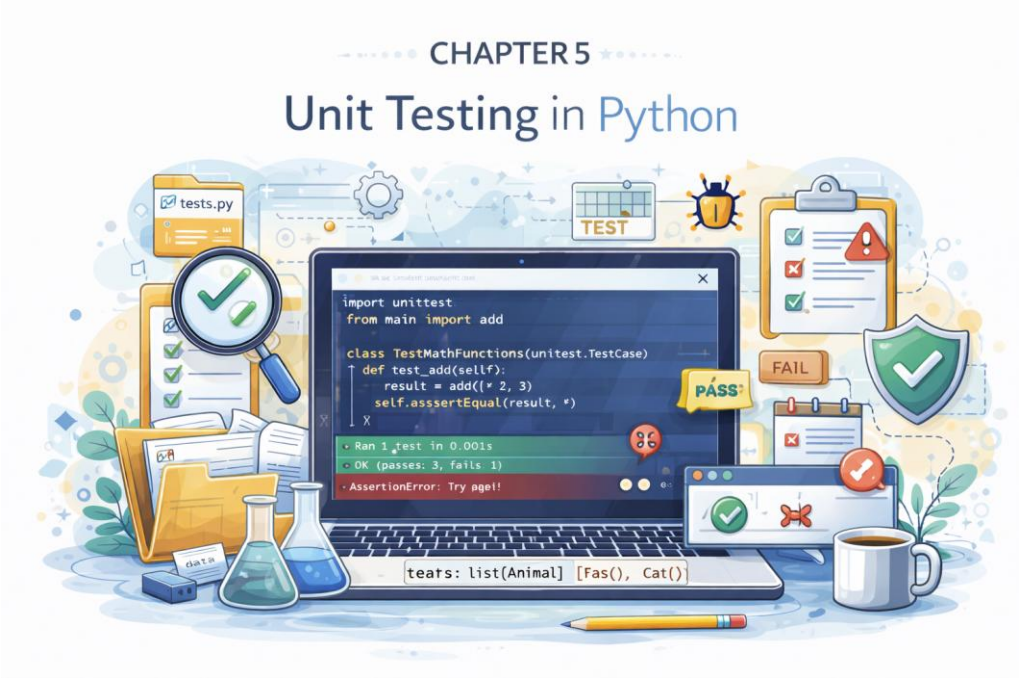

The following was generated using ChatGPT and further edited by Lucas Hartman

Chapter 5 — Unit Testing with Python's unittest

As programs grow larger, it becomes harder to be sure that every part still works the way we expect. One small change in one function can accidentally break something somewhere else — often in ways that aren't immediately obvious. That's why professional developers rely on unit testing: a systematic way to check that individual pieces of code, called units, behave correctly on their own. Each test runs automatically, verifying that a specific function or method produces the right result, even as the rest of the program evolves.

Unit testing is more than just "making sure it runs." It's about writing automated proofs that your code does what you think it does. When you modify or extend a program, your tests act like a safety net — catching mistakes before they reach users. Tests also serve as living documentation, showing how your functions and classes are intended to behave. They make refactoring less stressful and give you confidence to make improvements without fear of breaking existing features.

In this chapter, you'll learn how to use Python's built-in unittest framework to create and run your own tests. We'll start by writing simple tests for functions, then move on to testing objects, exceptions, and input/output. Along the way, you'll learn how to use common test structures and assertions, how to isolate external dependencies, and how to measure how much of your code is being tested. By the end of the chapter, you'll not only understand how to test but also why testing is an essential part of every well-designed software system.

5.1 What is Unit Testing (and Why Do We Care)?

Unit tests are small, automated checks that verify a tiny piece of code (a "unit"): usually a single function or method. They help you catch bugs early, refactor safely, and they act as living documentation for behavior. Good tests are fast, repeatable, and isolated—if a test depends on user input, files, or the network, we typically fake those bits, so the test stays focused on a single behavior.

Here we are going to use Python's built in module **`unittest`**. There are other libraries for testing but for now we are using the built-in module.

You will see that all the methods for testing are "assert" methods. For this the word assert helps us not just run code but "make a claim" and "verify it's true." In our everyday language, the word assert means to state something confidently as a fact. When we write a test, we're doing exactly that, asserting that our code behaves the way we expect it to.

A bit of context as well, the word "assert" is also a Python keyword existing outside of **`unittest`** framework. It is used for quick runtime checks.

5.2 The unittest Basics

Python's built-in **`unittest`** framework discovers tests in classes that inherit from **`TestCase`**, where each test method name starts with **`test_`**. Here's a minimal example showing assertions for integers and floats:

```
# test_math_utils.py
from unittest import TestCase

def add(a, b):
    return a + b

class TestAdd(TestCase):
    def test_add_integers(self):
        self.assertEqual(add(2, 3), 5)

    def test_add_floats(self):
        self.assertAlmostEqual(add(0.1, 0.2), 0.3, places=7)

if __name__ == "__main__":
    import unittest
    unittest.main()
```

5.2.1 Common Assertions

The following are the assert methods that you will want to use when building your unit tests:

- **`assertEquals(a, b)`** – values are equal (numbers, strings, lists, dataclasses with __eq__ defined, etc.)
- **`assertNotEqual(a, b)`** – values differ
- **`assertTrue(x)/assertFalse(x)`** – Boolean conditions
- **`assertIs(a, b)/assertIsNot(a, b)`** – Identity checks (same object).
- **`assertIsNone(x)/assertIsNotNone(x)`** – presence checks
- **`assertIn(member, container)/assertNotIn(member, container)`** – sees if the 'member' is in a container (like a list/dict)
- Order Relations
  - **`assertGreater(a, b)`** – checks that a > b
  - **`assertLess(a, b)`** – checks that a < b
  - **`assertGreaterEqual(a,b)`** – checks that a >= b
  - **`assertLessEqual(a,b)`** – checks that a <= b
- **`assertRaises(ExceptionType)`** – verify code raises an expected exception (see 5.7)
- **`assertAlmostEqual(a, b, places=#)`** – floating-point comparisons

Use the most specific assert available to produce helpful failure messages.

5.3 Test Discovery, File Layout, and Naming

It is important for testing and the automation of running the tests to keep them all together. Name test files as `test_*.py` or `*_test.py` and group them together in a package/folder like `test/test_utils.py`, etc.

You can then use the command line to run your tests

```
python -m unittest
python -m unittest discover
```

discover gives options that you may add.

It is important to use clear names so that not only does `unittest` find them that you know what each file is doing.

- Test class names: `Test<Thing>`
- Test method names: `test_<behavior>`
    - Each test should only test one component, the names can be as long as needed but describe what is being tested/how. You will see this in examples that follow.

5.4 Test Fixtures: setUp, tearDown, and Friends

Fixtures create a clean state for each test and then clean up. Use setUp for per-test initialization and tearDown for cleanup. Use these to set up objects, empty lists, etc. to use in your tests making sure they work.

The class methods, setUpClass/tearDownClass run once per class when you have expensive setup to share. Here the @classmethod is required as these methods are run on the class not per instance for testing. You will then have 'cls' as the variable to reference in these methods instead of 'self' when accessing values. Like setting up some objects to use in all tests. But you will still use 'self' to access the class variables in your test methods.

You don't need to delete objects/creation in the tearDownClass, this is more external resources. Like closing a file, a network connection, etc. as teardown.

```
from unittest import TestCase

class TestCart(TestCase):
    def setUp(self):
        self.items = []

    def tearDown(self):
        self.items.clear()

    def test_add_item(self):
        self.items.append("book")
        self.assertIn("book", self.items)

    @classmethod
    def setUpClass(cls):
        cls.catalog_loaded = True

    @classmethod
    def tearDownClass(cls):
        cls.catalog_loaded = False
```

5.5 Testing Pure Functions (Logic, Parsing, Formatting)

Start by testing pure functions (no I/O). They're easiest to test and encourage clean design. This example shows happy paths and an exception case using assertRaises:

```
# utils.py
def percent_change(old, new):
    if old == 0:
        raise ZeroDivisionError("old value cannot be zero")
    return ((new - old) / old) * 100

# test_utils.py
from unittest import TestCase
from utils import percent_change

class TestPercentChange(TestCase):
    def test_positive_growth(self):
        self.assertAlmostEqual(percent_change(100, 120), 20.0)

    def test_negative_growth(self):
        self.assertAlmostEqual(percent_change(200, 150), -25.0)

    def test_zero_old_raises(self):
        with self.assertRaises(ZeroDivisionError):
            percent_change(0, 10)
```

5.6 Testing Objects (Methods, __str__, and Behavior)

Here we test classes from the OOP chapter—using protected attributes, __str__ for representation, and subclass-specific behavior. We are testing both data and representations. Note each method name in the test class reads as you know what is being tested.

5.6.1 Objects to test

```
# employees.py
class Employee:
    def __init__(self, name, department):
        self._name = name
        self._department = department
    def __str__(self):
        return f"Name: {self._name}\nDepartment: {self._department}"

class SalariedEmployee(Employee):
    def __init__(self, name, department, annual_salary):
        super().__init__(name, department)
        self._annual_salary = annual_salary
    def weekly_pay(self) -> float:
        return self._annual_salary / 52
    def __str__(self):
        base = super().__str__()
        return f"{base}\nWeekly Pay: ${self.weekly_pay():.2f}"

class HourlyEmployee(Employee):
    def __init__(self, name, department, hourly_rate, hours_worked):
        super().__init__(name, department)
        self._hourly_rate = hourly_rate
        self._hours_worked = hours_worked
    def weekly_pay(self) -> float:
        return self._hourly_rate * self._hours_worked
    def __str__(self):
        base = super().__str__()
        return f"{base}\nWeekly Pay: ${self.weekly_pay():.2f}"
```

5.6.2 Unit Tests

```
# test_employees.py
from unittest import TestCase
from employees import Employee, SalariedEmployee, HourlyEmployee

class TestEmployeeHierarchy(TestCase):
    def test_str_base(self):
        e = Employee("Ava", "Support")
        self.assertIn("Name: Ava", str(e))
        self.assertIn("Department: Support", str(e))

    def test_salaried_weekly_pay(self):
        e = SalariedEmployee("Liam", "Management", 104000)  # $2000/week
        self.assertEqual(e.weekly_pay(), 2000)

    def test_hourly_weekly_pay(self):
        e = HourlyEmployee("Noah", "Sales", 20.0, 30)
        self.assertEqual(e.weekly_pay(), 600.0)
```

5.7 Testing Exceptions and Edge Cases

Use **`assertRaises`** as a context manager when you expect errors, and assert on the message for clarity so failures are informative:

```
def parse_percent(text: str) -> float:
    if not text.endswith("%"):
        raise ValueError("Missing %")
    return float(text[:-1])

from unittest import TestCase

class TestParsing(TestCase):
    def test_valid(self):
        self.assertAlmostEqual(parse_percent("12.5%"), 12.5)

    def test_missing_symbol(self):
        with self.assertRaises(ValueError) as cm:
            parse_percent("12.5")
        self.assertIn("Missing %", str(cm.exception))
```

5.8 Isolating I/O with unittest.mock (faking input/print)

When testing, it's best to avoid real user input or printed output – tests should run automatically without someone typing or reading anything. To accomplish this, Python's unittest.mock module lets us replace built-in functions like input() or print() during a test, using a temporary "fake" version.

This allows us to check how our code behaves without manual interaction. That is what unit testing is all about. By mocking these functions we can simulate user input automatically, capture printed output for verification and keep tests fast, repeatable and fully automated.

5.8.1 The @patch Decorator

The `@patch` decorator from `unittest.mock` replaces the real function (like `input()`) with a mock object that behaves as we define it.

This mock can return preset values or record what was called.

```
# app.py
def ask_name():
    name = input("Enter name: ")
    print(f"Hello, {name}!")
    return name

# test_app.py
from unittest import TestCase
from unittest.mock import patch
from app import ask_name

class TestApp(TestCase):
    @patch("builtins.input", return_value="Ava")
    @patch("builtins.print")
    def test_ask_name(self, mock_print, mock_input):
        self.assertEqual(ask_name(), "Ava")
        mock_print.assert_called_with("Hello, Ava!")
```

Here our **`@patch`** for **`builtins.input`** replaces the **`input()`** with a fake version by setting the **`return_value`** to what will be always returned.

The **`@patch`** for **`builtins.print`** replaces the print function with a mock so the test can verify what was printed

The order of the decorators and parameters here matters!

- The lowest decorator argument(**`mock_print`**) matches the first decorator listed above the function – **`builtins.print`**
- The next one matches the second decorators, and so on

5.9 Table-Driven Tests with subTest

unittest does not ship parameterized tests, but **`subTest`** gives a clean pattern to iterate through small tables while keeping failures specific:

```
from unittest import TestCase

def discount(price, rate):
    return price * (1 - rate)

class TestDiscount(TestCase):
    def test_table(self):
        cases = [
            (100, 0.10, 90),
            (200, 0.25, 150),
            ( 50, 0.00, 50),
        ]
        for price, rate, expected in cases:
            with self.subTest(price=price, rate=rate):
                self.assertEqual(discount(price, rate), expected)
```

5.10 Running Tests as You Work (and in CI)

Run all tests with **`python -m unittest -v`** for a verbose output

To only run a specific test on module, class, etc.

```
python -m unittest tests.test_employees.TestEmployeeHierarchy.test_salaried_weekly_pay
```

Later, you can measure coverage with `coverage.py` to see which parts of your code are exercised.

Test coverage measures how much of your program's code is actually executed when you run your tests. It basically answers, "When I run my tests, what percentage of my code is being checked?"

It's not about whether your tests are good, but about whether your tests are reaching the code that exists.

- Statement coverage – did every line of code run at least once?
- Branch coverage – did every if/else/conditional path run?
- Function coverage – did every function get called?
- Path coverage – did all possible code paths (combinations of conditions) execute?

While not really required for class if you would like to use coverage, it needs to be installed. It doesn't come preinstalled like `unittest`, we have to use our package installer, `pip`.

```
pip install coverage
```

You can run the above code in your command line in PyCharm or on your local command prompt/terminal to have it installed globally. If you run it in PyCharm local it will only install on the local environment and need to install each new project.

Once installed, use it from the command line to execute your tests and see percentages

```
coverage run -m unittest discover
```

```
coverage report -m
```

A word of caution, just because you have high coverage doesn't mean you have high quality tests. You should focus on the quality of tests over coverage for now.

5.11 Best Practices (Quick Checklist)

- Keep tests small, fast, and isolated.
- One behavior per test method; use descriptive names.
- Prefer pure functions for core logic; inject or mock I/O.
- Use the most specific assertion available.
- Test both happy paths and edge cases.
- Keep fixtures minimal; set up only what each test needs.
- Use subTest for table-driven scenarios.